SIMATS ENGINEERING

ASSESSMENT TOOL 3

Course: Design and Analysis of Algorithms (CSA06)

Course Outcome 4: Solve optimization problems using dynamic programming and greedy strategies.

Extended Comparative Analysis Document

Comprehensive Comparative Analysis

This document provides an in-depth comparative analysis of fifteen selected optimization problems. It critically evaluates the application of Dynamic Programming (DP) and Greedy Strategies, analyzing their correctness, computational complexity, and optimality across various constraints.

1. Knapsack Comparison (DP vs Greedy)

Context: Given items with weights [10, 20, 30] and values [60, 100, 120] with a capacity of 50.

0/1 Knapsack (Dynamic Programming): The DP approach builds a table to evaluate all possible item combinations without exceeding the weight limit. Since items cannot be fractioned, it checks the maximum value obtainable for every capacity up to 50. The optimal selection includes the 20-weight and 30-weight items, yielding a total value of 220. Time complexity is O(N*W), which can be computationally expensive for large capacities.

Fractional Knapsack (Greedy): The greedy strategy calculates the value-to-weight ratios (6, 5, and 4) and sorts items in descending order. It fully takes the 10-weight and 20-weight items, then takes a fraction (20/30) of the 30-weight item to fill the remaining capacity. The total value is 240. Time complexity is O(N log N) due to sorting.

Insights: Greedy is strictly optimal only when fractions are allowed, maximizing resource utilization perfectly. DP is necessary for the 0/1 constraint because the greedy choice might leave small, unfillable gaps in capacity, leading to a globally suboptimal outcome.

2. Shortest Path Comparison (Dijkstra vs Bellman-Ford)

Context: Finding the shortest path in a graph that includes negative edge weights.

Dijkstra's Algorithm (Greedy): Dijkstra extracts the minimum-distance unvisited node and relaxes its neighbors. It assumes that once a node is visited, its shortest path is permanently finalized. Because of this greedy choice property, if a subsequent path contains a negative weight that makes a previously "longer" route cheaper, Dijkstra fails to update the finalized node. Complexity is O(V^2) or O(E + V log V).

Bellman-Ford Algorithm (DP): Bellman-Ford relaxes all edges V-1 times. It uses the principle of optimality to iteratively refine the shortest paths, allowing it to correctly integrate negative weights. A final V-th relaxation pass allows it to detect negative weight cycles. Complexity is O(V * E).

Insights: Dijkstra is vastly superior in performance for routing protocols (like OSPF) where weights (distance, latency) are strictly non-negative. Bellman-Ford is mandatory for financial or specific theoretical networks where negative weights (representing gains or refunds) exist, despite its higher computational overhead.

3. MST Comparison (Prim vs Kruskal)

Context: Constructing a Minimum Spanning Tree (MST) from a weighted graph.

Prim's Algorithm: Prim's is a vertex-centric greedy algorithm. It initializes from an arbitrary node and continuously adds the minimum-weight edge that expands the tree to an unvisited node. Using an adjacency list and a Fibonacci heap, it achieves O(E + V log V) time.

Kruskal's Algorithm: Kruskal's is an edge-centric greedy algorithm. It sorts all edges in the entire graph globally and adds them one by one to the MST, provided they do not form a cycle (checked via a Disjoint Set Union data structure). Its complexity is dominated by sorting: O(E log E).

Insights: Both paradigms successfully yield the same optimal MST weight. Algorithm selection depends purely on graph density. Prim's algorithm is structurally better suited for highly dense graphs, whereas Kruskal's is generally faster and easier to implement for sparse graphs.

4. Coin Change Problem (Greedy vs DP)

Context: Given coin denominations [1, 3, 4] and an amount of 6, find the minimum number of coins.

Greedy Approach: The algorithm selects the largest denomination less than or equal to the remaining amount. For 6, it picks 4, leaving 2. It then picks 1, leaving 1, and finally picks another 1. Total coins: 3 (4 + 1 + 1).

Dynamic Programming: The DP algorithm formulates the recurrence: DP[i] = min(DP[i], DP[i - coin] + 1). It computes the minimum coins for every value from 1 to 6. For 6, it evaluates (4+1+1) yielding 3, and (3+3) yielding 2. The optimal answer is 2 coins.

Insights: Greedy fails spectacularly here because the coin set is non-canonical. The greedy choice (picking 4) traps the algorithm into a suboptimal branch. DP explores all subproblems, proving that while greedy is fast for standard currencies (like US coins), DP is universally required for arbitrary denomination sets.

5. Activity Selection (Greedy vs DP)

Context: Activities with start times [1, 3, 0, 5, 8, 5] and finish times [2, 4, 6, 7, 9, 9].

Greedy Strategy: The activities are sorted primarily by their finish times. The greedy choice asserts that picking the earliest finishing activity leaves the maximum possible time for remaining activities. It selects (1,2), then (3,4), then (5,7), and (8,9). Result: 4 activities. Complexity is O(N log N).

Dynamic Programming: A DP approach would require defining compatible subsets and evaluating the maximum activities in range subproblems (e.g., c[i, j] representing the max activities between the finish of i and start of j). This runs in O(N^3) or O(N^2) if optimized.

Insights: This is a classic example where the Greedy Choice Property inherently holds true. Choosing the earliest finisher never restricts future optimal choices in a way that yields a lesser total count. Using DP here is computationally wasteful and demonstrates over-engineering.

6. All-Pairs vs Single-Source Shortest Path

Context: Computing shortest paths using Floyd-Warshall and Dijkstra's algorithm.

Dijkstra (Single-Source): Dijkstra computes the shortest path from one specific starting node to all other nodes in O(V^2) or O(E + V log V). To find all-pairs shortest paths, Dijkstra must be run V times, resulting in O(V * E + V^2 log V) complexity.

Floyd-Warshall (All-Pairs DP): Floyd-Warshall operates on an adjacency matrix and considers every node as an intermediate point for every pair of nodes. Its nested three-loop structure results in a strict O(V^3) time complexity and O(V^2) space complexity.

Insights: If the graph is dense, Floyd-Warshall is incredibly elegant and easy to implement, and handles negative weights gracefully. However, for sparse graphs with non-negative weights, running Dijkstra V times is drastically faster than the cubic time required by Floyd-Warshall, highlighting the importance of scalability profiling.

7. String Matching (LCS vs Greedy)

Context: Longest Common Subsequence between "ABCBDAB" and "BDCAB".

Greedy Attempt: A greedy approach might scan the first string and immediately lock in the first matching character found in the second string. If it locks onto the first 'B', it might skip past a sequence that yields a longer overall match, often getting stuck in local optima and missing interleaved sequences.

Dynamic Programming: The DP approach constructs a 2D table to map overlapping subproblems, checking if characters match (adding 1 to the diagonal) or don't match (taking the max of the top or left cell). The result perfectly identifies "BDAB" or "BCAB" with length 4.

Insights: LCS possesses Optimal Substructure but completely lacks the Greedy Choice Property. The decision to include a character depends fundamentally on the remaining substrings on both sides. Greedy inevitably fails because character matching requires contextual foresight provided by DP tabulation.

8. TSP (DP vs Greedy Heuristic)

Context: Travelling Salesman Problem for 4 cities.

Greedy Heuristic (Nearest Neighbor): Starting from a city, the algorithm always visits the nearest unvisited city. While incredibly fast (O(N^2)), it often ends up trapped. The final required return edge might be astronomically expensive because all cheap edges were used early on.

Dynamic Programming (Held-Karp): The DP approach maps the shortest path visiting a specific subset of cities and ending at a specific city. It guarantees absolute optimality by exhaustively checking sub-paths, but it operates in exponential time: O(N^2 * 2^N).

Insights: The trade-off is stark: Exact optimality vs. Computability. For small N (like 4), DP is perfect. For N = 50, DP is computationally impossible due to state-space explosion, forcing engineers to rely on greedy heuristics or metaheuristics despite their sub-optimal nature.

9. Matrix Chain Multiplication (DP vs Greedy)

Context: Matrices (10x20), (20x30), (30x40). Compare multiplication order approaches.

Greedy Approach: A naive greedy approach might try to multiply the matrices with the smallest dimensions first to keep intermediate matrices small. However, local dimension choices easily lead to massive scalar multiplications in the final steps.

Dynamic Programming: DP tests every possible parenthesization split recursively (memoized). It calculates the cost of the left sub-chain, right sub-chain, and the merge cost. This O(N^3) algorithm guarantees the absolute minimum number of scalar multiplications.

Insights: Matrix Chain Multiplication is a quintessential DP problem because the cost of multiplication is highly non-linear relative to dimensions. Greedy fails because a cheap early multiplication can mandate an unavoidably expensive final multiplication. Optimal substructure must be mapped fully.

10. Huffman Coding vs Fixed-Length Coding

Context: Frequencies {A:5, B:9, C:12, D:13, E:16, F:45}.

Fixed-Length Coding: With 6 symbols, we require 3 bits per symbol (e.g., A=000, F=101). Total bits = (5+9+12+13+16+45) * 3 = 100 * 3 = 300 bits. This is inefficient as high-frequency symbols take up exactly as much space as rare ones.

Huffman Coding (Greedy): The algorithm builds a prefix tree by continually merging the two lowest-frequency nodes. 'F' (45%) gets a 1-bit code (e.g., 0). 'A' gets a 4-bit code. The weighted path length compresses the total size significantly (typically around 224 bits in this scenario).

Insights: Huffman coding is a flawless application of the greedy strategy for data compression. By ensuring the most common symbols have the shortest codes, it creates an optimally compressed prefix-free system that vastly outperforms fixed-length encoding in real-world bandwidth savings.

11. Coin Change (1, 4, 6) for Amount 8

Context: Denominations [1, 4, 6], target 8. Compare approaches.

Greedy Strategy: Picks 6, leaving 2. Then picks 1, leaving 1, and picks 1. Total: 3 coins (6, 1, 1).

DP Strategy: Evaluates all paths and identifies that picking two 4s yields exactly 8. Total: 2 coins (4, 4).

Insights: Once again, the arbitrary coin denominations expose the weakness of the greedy choice. Scalability of DP is O(amount * denominations), meaning it remains highly efficient for reasonable target amounts while ensuring currency systems without canonical properties still yield optimal change.

12. Knapsack (0/1 DP vs Fractional Greedy) Structural Review

Context: Weights [10, 20, 30], Values [60, 100, 120], Cap 50.

Evaluation: The greedy fraction approach achieves 240, while DP 0/1 achieves 220. DP requires O(NW) memory, meaning a matrix of size 3x50. If the capacity was 50,000, DP would face severe memory scaling issues compared to the O(1) space requirement of the Greedy algorithm.

Insights: When dealing with massive datasets, if items are discreet (indivisible) and capacity is huge, traditional DP fails due to memory constraints. This leads to the necessity of approximation algorithms or branch-and-bound techniques to balance the memory intensity of DP against the inaccuracy of pure Greedy.

13. Dice Throw Problem vs Coin Change

Context: Compare Dice Throw DP formulation with Coin Change.

Analysis: Both are variations of the partition problem. Coin change (optimization) seeks the *minimum* items, whereas Dice Throw (combinatorics) seeks the *total ways* to reach a sum. The state transition for Coin Change is min(DP[i], DP[i-c] + 1). The transition for Dice Throw is sum(DP[i-1][j-face]).

Insights: DP is incredibly versatile. By merely changing the operator from 'min' to 'sum', the structural paradigm shifts from resource optimization to combinatorial counting. Subproblem overlap remains the core mechanic enabling polynomial time execution.

14. Assembly Line vs Job Sequencing

Context: Compare DP in Assembly Line to Greedy in Job Sequencing.

Assembly Line (DP): Involves moving a chassis between two parallel factory lines. The optimal choice at station 'j' depends on the accumulated time to reach station 'j-1' on both lines, plus the transfer penalty. The optimal substructure requires looking at previous states.

Job Sequencing (Greedy): Given deadlines and profits, the algorithm sorts by descending profit and greedily assigns the job to the latest possible free slot before its deadline.

Insights: Job sequencing features independent decisions constrained only by time slots, fitting a Greedy approach perfectly. Assembly Line features *dependent* sequential state transitions where a transfer cost alters future pathways, strictly mandating Dynamic Programming.

15. Travelling Salesperson Problem (Scalability Deep Dive)

Context: Exact vs Heuristic methods for TSP.

Exponential Wall: The DP solution for TSP utilizes bitmasking to track visited cities. While it reduces the time complexity from factorial O(N!) to exponential O(N^2 * 2^N), it practically hits a wall around N=25. The space complexity O(N * 2^N) exhausts RAM very quickly.

Greedy Compromise: The Nearest Neighbor greedy approach operates in O(N^2), easily scaling to N=10,000 cities in milliseconds. While it does not guarantee optimality (often resulting in a tour 20-30% longer than the optimum), it scales endlessly.

Insights: TSP perfectly illustrates the boundaries of Dynamic Programming. DP provides exact correctness but fails scalability tests. Real-world logistics engines (like delivery routing) abandon pure DP in favor of greedy construction followed by local search optimizations (like 2-opt) to balance speed and route quality.